A Gentle Introduction to Neural Networks for Predictive Modeling

Owen Visser

University of Florida

October 17, 2026

Contents

1. The difference between predictive modeling and generative modeling
2. What does a neural network consist of?
3. Why activation functions matter.
4. How backpropagation works.
5. How powerful is this, really?

Generally speaking, there are two modeling goals...

Predictive modeling - *What happens when I have data?*

Given observed features $\boldsymbol{x}$, estimate an outcome y:

$$\widehat{y} = f_{\boldsymbol{\theta}}(\boldsymbol{x}) \qquad \text{or} \qquad p_{\boldsymbol{\theta}}(y|\boldsymbol{x}).$$

Generally speaking, there are two modeling goals...

Predictive modeling - *What happens when I have data?*

Given observed features $\boldsymbol{x}$, estimate an outcome y:

$$\widehat{y} = f_{\boldsymbol{\theta}}(\boldsymbol{x}) \qquad \text{or} \qquad p_{\boldsymbol{\theta}}(y|\boldsymbol{x}).$$

Predictions are ultimately represented as one or more scalar values.

Generally speaking, there are two modeling goals...

Predictive modeling - *What happens when I have data?*

Given observed features $\boldsymbol{x}$, estimate an outcome y:

$$\widehat{y} = f_{\boldsymbol{\theta}}(\boldsymbol{x}) \qquad \text{or} \qquad p_{\boldsymbol{\theta}}(y|\boldsymbol{x}).$$

Predictions are ultimately represented as one or more scalar values.

Generative modeling - *How to get the data to begin with?*

Model the data-generating distribution itself, often written as one of the following:

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}), \qquad p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid y), \qquad p_{\boldsymbol{\theta}}(\boldsymbol{x}, y).$$

Generally speaking, there are two modeling goals...

Predictive modeling - *What happens when I have data?*

Given observed features $\boldsymbol{x}$, estimate an outcome y:

$$\widehat{y} = f_{\boldsymbol{\theta}}(\boldsymbol{x}) \qquad \text{or} \qquad p_{\boldsymbol{\theta}}(y|\boldsymbol{x}).$$

Predictions are ultimately represented as one or more scalar values.

Generative modeling - *How to get the data to begin with?*

Model the data-generating distribution itself, often written as one of the following:

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}), \qquad p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid y), \qquad p_{\boldsymbol{\theta}}(\boldsymbol{x}, y).$$

Construct a model capable of generating plausible $\boldsymbol{x}$ from the learned distribution.

Generally speaking, there are two modeling goals...

Predictive modeling - *What happens when I have data?*

Given observed features $\boldsymbol{x}$, estimate an outcome y:

$$\widehat{y} = f_{\boldsymbol{\theta}}(\boldsymbol{x}) \qquad \text{or} \qquad p_{\boldsymbol{\theta}}(y|\boldsymbol{x}).$$

Predictions are ultimately represented as one or more scalar values.

Generative modeling - *How to get the data to begin with?*

Model the data-generating distribution itself, often written as one of the following:

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}), \qquad p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid y), \qquad p_{\boldsymbol{\theta}}(\boldsymbol{x}, y).$$

Construct a model capable of generating plausible $\boldsymbol{x}$ from the learned distribution.
Often accomplished using predictive neural networks as components of the larger generative procedure.

$$p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid f_{\boldsymbol{\theta}}(\boldsymbol{x}))$$

Synergy of simple procedures

Consider ordinary linear regression:

$$\widehat{y}_i = \sum_{j=1}^{n} x_{ij}\omega_j + \beta, \qquad \text{often written as} \qquad \widehat{y}_i = \boldsymbol{W}\boldsymbol{x}_i + \beta.$$

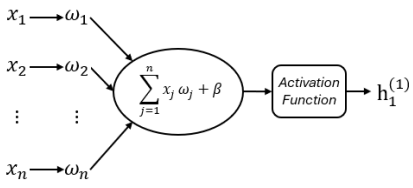
where ω_j are weights and n is the number of features.

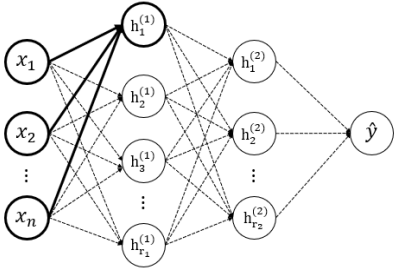
Synergy of simple procedures

Consider ordinary linear regression:

$$\widehat{y}_i = \sum_{j=1}^{n} x_{ij}\omega_j + \beta, \qquad \text{often written as} \qquad \widehat{y}_i = \boldsymbol{W}\boldsymbol{x}_i + \beta.$$

where ω_j are weights and n is the number of features.
Neural Networks are created from multiple ordinary linear regressions:



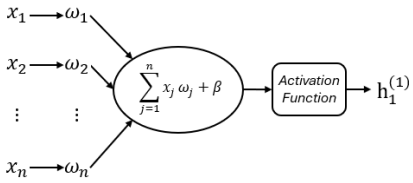


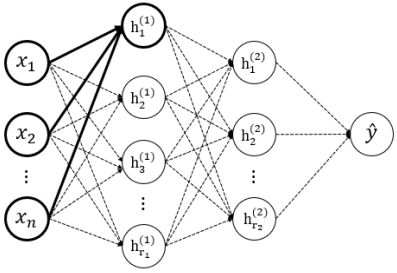
Synergy of simple procedures

Consider ordinary linear regression:

$$\widehat{y}_i = \sum_{j=1}^{n} x_{ij}\omega_j + \beta, \qquad \text{often written as} \qquad \widehat{y}_i = \boldsymbol{W}\boldsymbol{x}_i + \beta.$$

where ω_j are weights and n is the number of features.
Neural Networks are created from multiple ordinary linear regressions:





Wait, **what is an activation function?**

Without the activation function, it all collapses

Suppose we combine two neural network layers, where the first and second layer are

$$\boldsymbol{h}^{(1)} = \boldsymbol{W}_1^\top \boldsymbol{x} + \boldsymbol{\beta}_1 \qquad \text{and} \qquad \boldsymbol{h}^{(2)} = \boldsymbol{W}_2^\top \boldsymbol{h}^{(1)} + \boldsymbol{\beta}_2.$$

Without the activation function, it all collapses

Suppose we combine two neural network layers, where the first and second layer are

$$\boldsymbol{h}^{(1)} = \boldsymbol{W}_1^\top \boldsymbol{x} + \boldsymbol{\beta}_1 \qquad \text{and} \qquad \boldsymbol{h}^{(2)} = \boldsymbol{W}_2^\top \boldsymbol{h}^{(1)} + \boldsymbol{\beta}_2.$$

Substituting the first layer equation into the second gives

$$\boldsymbol{h}^{(2)} = \boldsymbol{W}_2^\top (\boldsymbol{W}_1^\top \boldsymbol{x} + \boldsymbol{\beta}_1) + \boldsymbol{\beta}_2.$$

Without the activation function, it all collapses

Suppose we combine two neural network layers, where the first and second layer are

$$\boldsymbol{h}^{(1)} = \boldsymbol{W}_1^\top \boldsymbol{x} + \boldsymbol{\beta}_1 \qquad \text{and} \qquad \boldsymbol{h}^{(2)} = \boldsymbol{W}_2^\top \boldsymbol{h}^{(1)} + \boldsymbol{\beta}_2.$$

Substituting the first layer equation into the second gives

$$\boldsymbol{h}^{(2)} = \boldsymbol{W}_2^\top (\boldsymbol{W}_1^\top \boldsymbol{x} + \boldsymbol{\beta}_1) + \boldsymbol{\beta}_2.$$

This can be represented simply by a single layer with a new weight matrix:

$$\boldsymbol{h}^{(2)} = (\boldsymbol{W}^*)^\top \boldsymbol{x} + \boldsymbol{\beta}^* \qquad \text{where} \qquad \boldsymbol{W}^* = \boldsymbol{W}_1 \boldsymbol{W}_2, \quad \boldsymbol{\beta}^* = \boldsymbol{W}_2^\top \boldsymbol{\beta}_1 + \boldsymbol{\beta}_2.$$

No matter how many linear layers we stack, they collapse into a **single linear layer**.

Without the activation function, it all collapses

Suppose we combine two neural network layers, where the first and second layer are

$$\boldsymbol{h}^{(1)} = \boldsymbol{W}_1^\top \boldsymbol{x} + \boldsymbol{\beta}_1 \qquad \text{and} \qquad \boldsymbol{h}^{(2)} = \boldsymbol{W}_2^\top \boldsymbol{h}^{(1)} + \boldsymbol{\beta}_2.$$

Substituting the first layer equation into the second gives

$$\boldsymbol{h}^{(2)} = \boldsymbol{W}_2^\top (\boldsymbol{W}_1^\top \boldsymbol{x} + \boldsymbol{\beta}_1) + \boldsymbol{\beta}_2.$$

This can be represented simply by a single layer with a new weight matrix:

$$\boldsymbol{h}^{(2)} = (\boldsymbol{W}^*)^\top \boldsymbol{x} + \boldsymbol{\beta}^* \qquad \text{where} \qquad \boldsymbol{W}^* = \boldsymbol{W}_1 \boldsymbol{W}_2, \quad \boldsymbol{\beta}^* = \boldsymbol{W}_2^\top \boldsymbol{\beta}_1 + \boldsymbol{\beta}_2.$$

No matter how many linear layers we stack, they collapse into a **single linear layer**.

The solution is... Activation Functions

Define a **non-linear** function, say $g(a)$, which will prevent the collapse:

$$\boldsymbol{h}^{(2)} = g(\boldsymbol{W}_2^\top \cdot g(\boldsymbol{W}_1^\top \boldsymbol{x} + \boldsymbol{\beta}_1) + \boldsymbol{\beta}_2).$$

Activation functions: what they do

Common activation functions

$$\mathrm{ReLU}(a) = \max(0, a), \qquad \sigma(a) = \frac{1}{1 + e^{-a}}, \qquad \tanh(a) = \frac{e^{a} - e^{-a}}{e^{a} + e^{-a}}.$$

Activation functions should be **cheap**, "almost everywhere" **differentiable**, and **stable**.

ReLU is popular, as it avoids **saturation** and creates sparse activations.

Historically, AlexNet helped popularize deep convolutional networks in 2012; ReLU and GPU training were important practical ingredients.

The problem with saturation

Sigmoid has derivative

$$\sigma'(a) = \sigma(a)\{1 - \sigma(a)\}.$$

When a is very negative or very positive we call the gradient *saturated*,

$$\sigma'(a) \approx 0.$$

The problem with saturation

Sigmoid has derivative

$$\sigma'(a) = \sigma(a)\{1 - \sigma(a)\}.$$

When a is very negative or very positive we call the gradient *saturated*,

$$\sigma'(a) \approx 0.$$

Under gradient descent,

$$\boldsymbol{\theta}_1 = \boldsymbol{\theta}_0 - \eta \nabla_{\boldsymbol{\theta}_0} \mathcal{L}.$$

When the gradient becomes saturated the new parameter does not update,

$$\nabla_{\boldsymbol{\theta}_0} \mathcal{L} \approx 0 \quad \Longrightarrow \quad \boldsymbol{\theta}_1 \approx \boldsymbol{\theta}_0.$$

The problem with saturation

Sigmoid has derivative

$$\sigma'(a) = \sigma(a)\{1 - \sigma(a)\}.$$

When a is very negative or very positive we call the gradient *saturated*,

$$\sigma'(a) \approx 0.$$

Under gradient descent,

$$\boldsymbol{\theta}_1 = \boldsymbol{\theta}_0 - \eta \nabla_{\boldsymbol{\theta}_0} \mathcal{L}.$$

When the gradient becomes saturated the new parameter does not update,

$$\nabla_{\boldsymbol{\theta}_0} \mathcal{L} \approx 0 \quad \implies \quad \boldsymbol{\theta}_1 \approx \boldsymbol{\theta}_0.$$

Vanishing Gradient Problem

In deep networks, this can occur early on and compound across layers, resulting in a model that fails train its parameters.

Let's talk about training

For data $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$, training usually means solving for the maxima or minima of some convex function:

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \left[\frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\{f_{\boldsymbol{\theta}}(\boldsymbol{x}_i), y_i\} + \lambda \Omega(\boldsymbol{\theta}) \right],$$

where $\mathcal{L}$ is the loss function, $\Omega(\boldsymbol{\theta})$ is a regularization penalty with parameter λ, and $\boldsymbol{\theta}$ contains all model weights and biases.

Let's talk about training

For data $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$, training usually means solving for the maxima or minima of some convex function:

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \left[\frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\{f_{\boldsymbol{\theta}}(\boldsymbol{x}_i), y_i\} + \lambda \Omega(\boldsymbol{\theta}) \right],$$

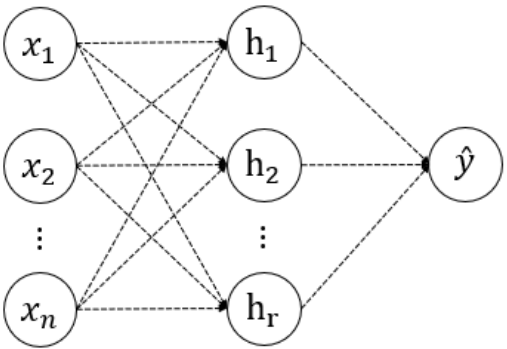
where $\mathcal{L}$ is the loss function, $\Omega(\boldsymbol{\theta})$ is a regularization penalty with parameter λ, and $\boldsymbol{\theta}$ contains all model weights and biases.

Familiar example: ridge regression

For the linear model $f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) = \beta_0 + \boldsymbol{x}_i^\top \boldsymbol{\beta}$, ridge regression solves

$$(\widehat{\beta}_0, \widehat{\boldsymbol{\beta}}) = \arg\min_{\beta_0, \boldsymbol{\beta}} \left[\frac{1}{N} \sum_{i=1}^{N} \left(y_i - \beta_0 - \boldsymbol{x}_i^\top \boldsymbol{\beta} \right)^2 + \lambda \|\boldsymbol{\beta}\|_2^2 \right].$$

Theoretical example: one hidden layer network

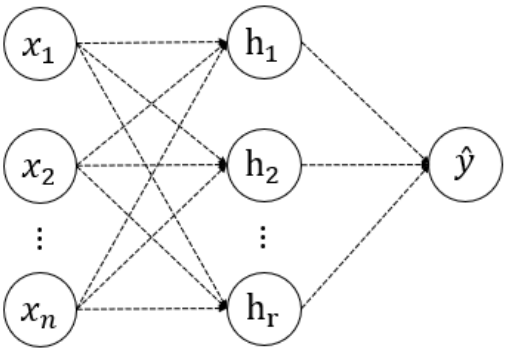


A single hidden layer network is mathematically defined as

$$\boldsymbol{h} = g(\boldsymbol{a}) = g\left(\boldsymbol{W}_1^\top \boldsymbol{x} + \boldsymbol{\beta}_1\right),$$

$$\widehat{y} = \boldsymbol{w}_2^\top \boldsymbol{h} + \beta_2.$$

Theoretical example: one hidden layer network



A single hidden layer network is mathematically defined as

$$\boldsymbol{h} = g(\boldsymbol{a}) = g\left(\boldsymbol{W}_1^\top \boldsymbol{x} + \boldsymbol{\beta}_1\right),$$

$$\widehat{y} = \boldsymbol{w}_2^\top \boldsymbol{h} + \beta_2.$$

Here the trainable parameters are

$$\boldsymbol{\theta} = \{\boldsymbol{W}_1, \boldsymbol{\beta}_1, \boldsymbol{w}_2, \beta_2\},$$

and we will use squared error loss for the scalar output.

$$\mathcal{L} = \frac{1}{2}(\widehat{y} - y)^2$$

Backpropagation: starting at the loss

Just like gradient descent, we start with the loss and use the chain rule to reach the parameter we want to update. Backpropagation uses δ notation to simplify this process.

$$\mathcal{L} = \frac{1}{2}(\widehat{y} - y)^2 \quad \Longrightarrow \quad \delta^{(2)} := \frac{\partial \mathcal{L}}{\partial \widehat{y}} = \widehat{y} - y.$$

Since

$$\widehat{y} = \boldsymbol{w}_2^\top \boldsymbol{h} + \beta_2,$$

Backpropagation: starting at the loss

Just like gradient descent, we start with the loss and use the chain rule to reach the parameter we want to update. Backpropagation uses δ notation to simplify this process.

$$\mathcal{L} = \frac{1}{2}(\widehat{y} - y)^2 \quad \Longrightarrow \quad \delta^{(2)} := \frac{\partial \mathcal{L}}{\partial \widehat{y}} = \widehat{y} - y.$$

Since

$$\widehat{y} = \boldsymbol{w}_2^\top \boldsymbol{h} + \beta_2,$$

the output-layer gradients follow directly from the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{w}_2} = \frac{\partial \mathcal{L}}{\partial \widehat{y}} \cdot \frac{\partial \widehat{y}}{\partial \boldsymbol{w}_2} = \delta^{(2)} \boldsymbol{h} \quad \rightarrow \quad \boldsymbol{w}_2^{(\text{new})} = \boldsymbol{w}_2^{(\text{old})} - \eta \delta^{(2)} \boldsymbol{h},$$

$$\frac{\partial \mathcal{L}}{\partial \beta_2} = \frac{\partial \mathcal{L}}{\partial \widehat{y}} \cdot \frac{\partial \widehat{y}}{\partial \beta_2} = \delta^{(2)} \quad \rightarrow \quad \beta_2^{(\text{new})} = \beta_2^{(\text{old})} - \eta \delta^{(2)}.$$

Note: η here is a parameter called the "learning rate".

Backpropagation: move to the hidden layer

To update the first layer, we want to construct a $\boldsymbol{\delta}^{(1)}$ so simplify the complex notation. Let

$$\boldsymbol{\delta}^{(1)} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{a}} = \frac{\partial \mathcal{L}}{\partial \widehat{y}} \cdot \frac{\partial \widehat{y}}{\partial \boldsymbol{h}} \cdot \frac{\partial \boldsymbol{h}}{\partial \boldsymbol{a}} = \delta^{(2)} \cdot \boldsymbol{w}_2^{\top} \cdot \text{diag}\{g'(\boldsymbol{a})\}$$

Backpropagation: move to the hidden layer

To update the first layer, we want to construct a $\boldsymbol{\delta}^{(1)}$ so simplify the complex notation. Let

$$\boldsymbol{\delta}^{(1)} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{a}} = \frac{\partial \mathcal{L}}{\partial \widehat{y}} \cdot \frac{\partial \widehat{y}}{\partial \boldsymbol{h}} \cdot \frac{\partial \boldsymbol{h}}{\partial \boldsymbol{a}} = \delta^{(2)} \cdot \boldsymbol{w}_2^\top \cdot \text{diag}\{g'(\boldsymbol{a})\}$$

We can see that when we extend the partial to a parameter within $\boldsymbol{a} = \boldsymbol{W}_1^\top \boldsymbol{x} + \boldsymbol{\beta}_1$, we get a much easier equation to work with:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}_1} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{a}} \cdot \frac{\partial \boldsymbol{a}}{\partial \boldsymbol{W}_1} = \boldsymbol{x} \left(\boldsymbol{\delta}^{(1)}\right)^\top \quad \rightarrow \quad \boldsymbol{W}_1^{(\text{new})} = \boldsymbol{W}_1^{(\text{old})} - \eta \boldsymbol{x} \left(\boldsymbol{\delta}^{(1)}\right)^\top,$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\beta}_1} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{a}} \cdot \frac{\partial \boldsymbol{a}}{\partial \boldsymbol{\beta}_1} = \boldsymbol{\delta}^{(1)} \quad \rightarrow \quad \boldsymbol{\beta}_1^{(\text{new})} = \boldsymbol{\beta}_1^{(\text{old})} - \eta \boldsymbol{\delta}^{(1)}.$$

General formulation of backpropagation

For layer ℓ, define the linear component

$$\boldsymbol{a}^{(\ell)} = \text{linear function of the layer inputs and parameters,}$$

and define

$$\boldsymbol{\delta}^{(\ell)} := \frac{\partial \mathcal{L}}{\partial \boldsymbol{a}^{(\ell)}}.$$

General formulation of backpropagation

For layer ℓ, define the linear component

$$\boldsymbol{a}^{(\ell)} = \text{linear function of the layer inputs and parameters,}$$

and define

$$\boldsymbol{\delta}^{(\ell)} := \frac{\partial \mathcal{L}}{\partial \boldsymbol{a}^{(\ell)}}.$$

For any parameter θ in layer ℓ, the chain rule gives

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{a}^{(\ell)}} \frac{\partial \boldsymbol{a}^{(\ell)}}{\partial \theta} = \boldsymbol{\delta}^{(\ell)} \frac{\partial \boldsymbol{a}^{(\ell)}}{\partial \theta}.$$

General formulation of backpropagation

For layer ℓ, define the linear component

$$\boldsymbol{a}^{(\ell)} = \text{linear function of the layer inputs and parameters,}$$

and define

$$\boldsymbol{\delta}^{(\ell)} := \frac{\partial \mathcal{L}}{\partial \boldsymbol{a}^{(\ell)}}.$$

For any parameter θ in layer ℓ, the chain rule gives

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{a}^{(\ell)}} \frac{\partial \boldsymbol{a}^{(\ell)}}{\partial \theta} = \boldsymbol{\delta}^{(\ell)} \frac{\partial \boldsymbol{a}^{(\ell)}}{\partial \theta}.$$

Hence, gradient descent updates a the parameters of layer ℓ by

$$\theta_\ell^{(\text{new})} = \theta_\ell^{(\text{old})} - \eta \boldsymbol{\delta}^{(\ell)} \frac{\partial \boldsymbol{a}^{(\ell)}}{\partial \theta}.$$

Backpropagation computes $\boldsymbol{\delta}^{(\ell)}$ and propagates it backward through the network.

Returning to the Vanishing Gradient Problem

Expanding the parameter update gives

$$\theta_\ell^{(\text{new})} = \theta_\ell^{(\text{old})} - \eta \underbrace{\boldsymbol{\delta}^{(L)} \boldsymbol{W}_L^\top \operatorname{diag}(g'(\boldsymbol{a}^{(L-1)})) \cdots \boldsymbol{W}_{\ell+1}^\top \operatorname{diag}(g'(\boldsymbol{a}^{(\ell)}))}_{\text{Gradient propagation across layers}} \frac{\partial \boldsymbol{a}^{(\ell)}}{\partial \theta_\ell}.$$

Returning to the Vanishing Gradient Problem

Expanding the parameter update gives

$$\theta_\ell^{(\text{new})} = \theta_\ell^{(\text{old})} - \eta \underbrace{\boldsymbol{\delta}^{(L)} \boldsymbol{W}_L^\top \operatorname{diag}(g'(\boldsymbol{a}^{(L-1)})) \cdots \boldsymbol{W}_{\ell+1}^\top \operatorname{diag}(g'(\boldsymbol{a}^{(\ell)}))}_{\text{Gradient propagation across layers}} \frac{\partial \boldsymbol{a}^{(\ell)}}{\partial \theta_\ell}.$$

Vanishing gradients can arise from repeated applications of the chain rule or from flat regions of an objective function.

- **Iterative optimization:** As we have just seen.
- **Dynamical systems:** Sensitivity to initial conditions decays in contractive systems.
- **Variational inference:** Parameter saturation or flat regions of the KL divergence can produce negligible gradients before convergence.

Returning to the Vanishing Gradient Problem

Expanding the parameter update gives

$$\theta_\ell^{(\text{new})} = \theta_\ell^{(\text{old})} - \eta \underbrace{\boldsymbol{\delta}^{(L)} \boldsymbol{W}_L^\top \operatorname{diag}(g'(\boldsymbol{a}^{(L-1)})) \cdots \boldsymbol{W}_{\ell+1}^\top \operatorname{diag}(g'(\boldsymbol{a}^{(\ell)}))}_{\text{Gradient propagation across layers}} \frac{\partial \boldsymbol{a}^{(\ell)}}{\partial \theta_\ell}.$$

Vanishing gradients can arise from repeated applications of the chain rule or from flat regions of an objective function.

- **Iterative optimization:** As we have just seen.
- **Dynamical systems:** Sensitivity to initial conditions decays in contractive systems.
- **Variational inference:** Parameter saturation or flat regions of the KL divergence can produce negligible gradients before convergence.

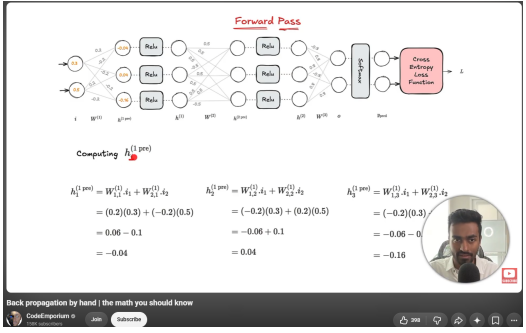
Common solutions include *residual connections*, *gated architectures*, and simply **lower depth**.

Practice makes perfect

Back propagation is the most important part of neural networks.

Try it out for yourself!

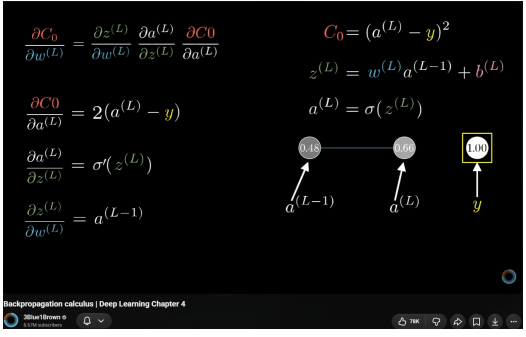
CodeEmporium



`https://youtu.be/12-HUfbyGso`

Consider a theoretical view!

3Blue1Brown



`https://youtu.be/tIeHLnjs5U8`

Improving on the most expensive step through batching

Vanilla back propagation is not used in practice.

- **Very slow** – Must calculate the full Jacobian for each backpropagation.
- **Often intractable** – due to memory requirements of the above.

The solution?

Improving on the most expensive step through batching

Vanilla back propagation is not used in practice.

- **Very slow** – Must calculate the full Jacobian for each backpropagation.
- **Often intractable** – due to memory requirements of the above.

The solution?

- **Stochastic Gradient Descent** (SGD) – uses *mini-batch samples* from the data to calculate a estimated Jacobian.

$$\boldsymbol{\theta}^{(\text{new})} = \boldsymbol{\theta}^{(\text{old})} - \eta \cdot \nabla_{\boldsymbol{\theta}} J\left(\boldsymbol{\theta}^{(\text{old})}; \boldsymbol{x}^{(i:i+n)}, \boldsymbol{y}^{(i:i+n)}\right)$$

The idea of using samples to calculate the Jacobian is shared among all practical **optimizers**.

Improving on the most expensive step through batching

Vanilla back propagation is not used in practice.

- **Very slow** – Must calculate the full Jacobian for each backpropagation.
- **Often intractable** – due to memory requirements of the above.

The solution?

- **Stochastic Gradient Descent** (SGD) – uses *mini-batch samples* from the data to calculate a estimated Jacobian.

$$\boldsymbol{\theta}^{(\text{new})} = \boldsymbol{\theta}^{(\text{old})} - \eta \cdot \nabla_{\boldsymbol{\theta}} J\left(\boldsymbol{\theta}^{(\text{old})}; \boldsymbol{x}^{(i:i+n)}, \boldsymbol{y}^{(i:i+n)}\right)$$

The idea of using samples to calculate the Jacobian is shared among all practical **optimizers**.

Unfortunately SGD gets stuck in local minima often, so it needs some **momentum** to get out of the hole.

Optimizing gradient descent with momentum

Consider the idea of "descent with memory", like a moving average:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \boldsymbol{v}_t$$

with

$$\boldsymbol{v}_t = \gamma \boldsymbol{v}_{t-1} + \eta \boldsymbol{g}_t, \qquad \text{and} \qquad \boldsymbol{g}_t = \nabla_{\boldsymbol{\theta}_t} J(\boldsymbol{\theta}_t),$$

where $\boldsymbol{v}_t$ is the *momentum*, and $\boldsymbol{g}_t$ is the gradient from the propagation, at time t. We can adjust γ and $\boldsymbol{v}$ to accelerate or attenuate the rate.

Optimizing gradient descent with momentum

Consider the idea of "descent with memory", like a moving average:

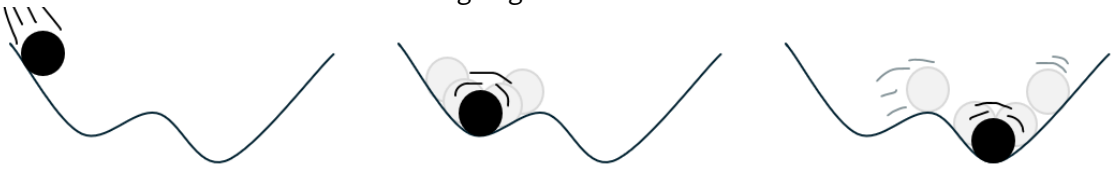
$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \boldsymbol{v}_t$$

with

$$\boldsymbol{v}_t = \gamma \boldsymbol{v}_{t-1} + \eta \boldsymbol{g}_t, \qquad \text{and} \qquad \boldsymbol{g}_t = \nabla_{\boldsymbol{\theta}_t} J(\boldsymbol{\theta}_t),$$

where $\boldsymbol{v}_t$ is the *momentum*, and $\boldsymbol{g}_t$ is the gradient from the propagation, at time t. We can adjust γ and $\boldsymbol{v}$ to accelerate or attenuate the rate.

The ball "remembers" it was going fast in order to avoid a local minimum.



Initial gradient descent | Without momentum | With momentum

The most common modern optimizers

The most common adaptive gradient is *Adaptive Moment Estimation* (Adam) (Kingma, 2015).

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon}\hat{\boldsymbol{m}}_t$$

where

$$\hat{\boldsymbol{m}}_t = \frac{\boldsymbol{m}_t}{1-\beta_1^t}, \qquad \boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1)\boldsymbol{g}_t, \qquad 0 \leq \beta_1 < 1,$$

$$\hat{\boldsymbol{v}}_t = \frac{\boldsymbol{v}_t}{1-\beta_2^t}, \qquad \boldsymbol{v}_t = \beta_1 \boldsymbol{v}_{t-1} + (1-\beta_2)\boldsymbol{g}_t^2, \qquad 0 \leq \beta_2 < 1,$$

$$\boldsymbol{g}_t = \nabla_{\boldsymbol{\theta}_t} J(\boldsymbol{\theta}_t),$$

and ϵ is a bounding constant.

The most common modern optimizers

The most common adaptive gradient is *Adaptive Moment Estimation* (Adam) (Kingma, 2015).

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon} \hat{\boldsymbol{m}}_t$$

where

$$\hat{\boldsymbol{m}}_t = \frac{\boldsymbol{m}_t}{1 - \beta_1^t}, \qquad \boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1 - \beta_1)\boldsymbol{g}_t, \qquad 0 \leq \beta_1 < 1,$$

$$\hat{\boldsymbol{v}}_t = \frac{\boldsymbol{v}_t}{1 - \beta_2^t}, \qquad \boldsymbol{v}_t = \beta_1 \boldsymbol{v}_{t-1} + (1 - \beta_2)\boldsymbol{g}_t^2, \qquad 0 \leq \beta_2 < 1,$$

$$\boldsymbol{g}_t = \nabla_{\boldsymbol{\theta}_t} J(\boldsymbol{\theta}_t),$$

and ϵ is a bounding constant.

At current time, Adam is generally the best optimizer, but the optimal choice is specific to the task (see "Comparison between Adam, Adamax, AdamW..." Llugsi, 2021)

Neural Networks in Practice

- **Python** has become the dominant language for developing and training neural networks.

Neural Networks in Practice

- **Python** has become the dominant language for developing and training neural networks.

- **PyTorch** provides a flexible framework for:
  - Constructing neural network architectures.
  - Automatically computing gradients through backpropagation.
  - Optimizing model parameters using SGD, Adam, etc.

Neural Networks in Practice

- **Python** has become the dominant language for developing and training neural networks.
- **PyTorch** provides a flexible framework for:
  - Constructing neural network architectures.
  - Automatically computing gradients through backpropagation.
  - Optimizing model parameters using SGD, Adam, etc.
- **GPU acceleration** allows PyTorch to efficiently parallelize matrix operations, substantially reducing training time for sufficiently large workloads.

Building a Neural Network

- Define a network using `nn.Module`.
- Initialize two linear layers with a ReLU activation.
- The forward pass:
  1. Applies the first linear transformation.
  2. Introduces nonlinearity.
  3. Produces the output logits.

```
import torch
import torch.nn as nn

class OneHiddenLayerNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(input, hidden)
        self.relu = nn.ReLU()
        self.output = nn.Linear(hidden, output)

    def forward(self, x):
        a = self.hidden(x)
        h = self.relu(a)
        logits = self.output(h)
        return logits
```

Training a Neural Network: Initialization

- Initialize the neural network.
- Select SGD as our optimizer.
- Define the classification loss.
- Introduce an adaptive learning rate.
- Load our training and validation data.

```
def train_model():
    model = OneHiddenLayerNN()

    optimizer = torch.optim.SGD(
        model.parameters(), lr=0.01
    )

    loss_fn = nn.CrossEntropyLoss()

    scheduler = ReduceLROnPlateau(
        optimizer, patience=3
    )

    train_loader, valid_loader = load_data()
```

Training a Neural Network: Each Epoch

- For each mini-batch:
  1. Reset the gradients.
  2. Compute predictions.
  3. Calculate the loss.
  4. Backpropagate.
  5. Update model weights.
- After each epoch:
  - Evaluate validation loss.
  - Adjust the learning rate.
  - Stop if performance stagnates.

```
for epoch in range(50):
    model.train()

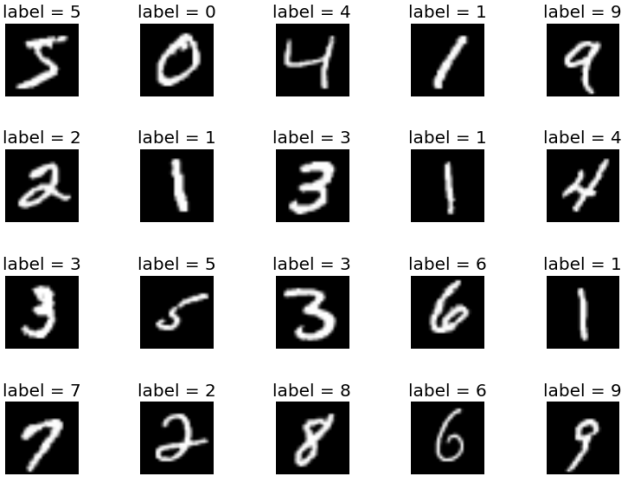
    for x, y in train_loader:
        optimizer.zero_grad()
        logits = model(x)
        loss = loss_fn(logits, y)

        loss.backward()
        optimizer.step()

    valid_loss = evaluate(model, valid_loader)
    scheduler.step(valid_loss)

    if no_improvement_for_10_epochs:
        break
```

A toy example: MNIST



MNIST dataset (Deng et al., 2012)

Settings for the MNIST example

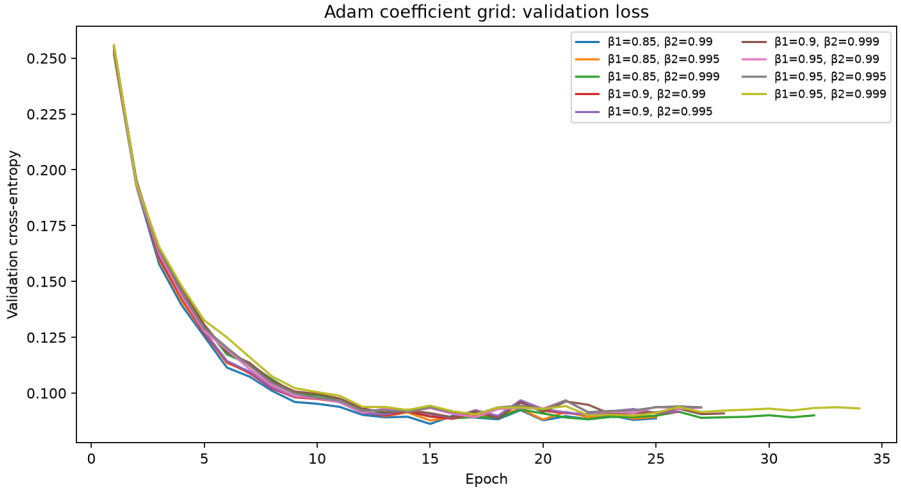
```
SEED = 42                       # set a random seed

INPUT_DIM = 784                 # input layer
HIDDEN_DIM = 64                 # hidden layer
OUTPUT_DIM = 10                 # output layer
BATCH_SIZE = 128                # number of samples we process at a time
EPOCHS = 100                    # Maximum; training may stop earlier

SGD_LR = 0.01                       # SGD learning rate
ADAM_DEFAULT_LR = 0.001             # Default ADAM learning rate
ADAM_DEFAULT_BETAS = (0.9,0.999)    # Default ADAM betas
ADAM_GRID = {                       # Grid of ADAM parameters (betas) we plan to check
    "beta1": [0.85, 0.9, 0.95],
    "beta2": [0.99, 0.995, 0.999]
    }

LR_SCHEDULER_FACTOR = 0.5       # learning rate adjustment from scheduler
LR_SCHEDULER_PATIENCE = 3       # how many "failed improvements" to wait before adjusting
EARLY_STOPPING_PATIENCE = 10    # how long before we stop entirely
```

Adam Hyperparameter Grid Loss Comparison



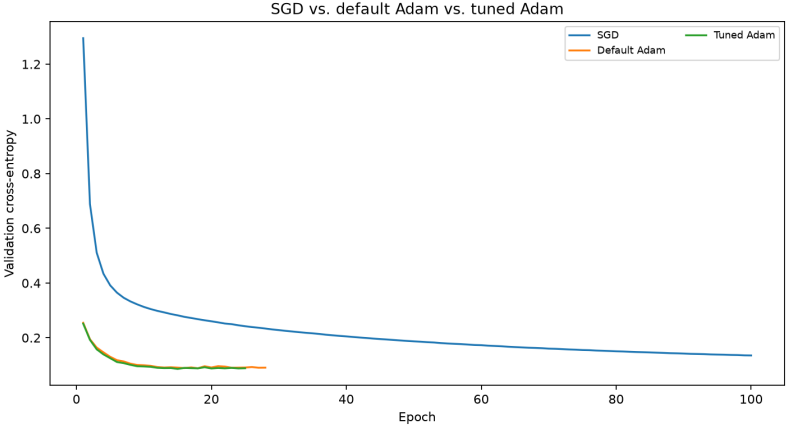
Hard to tell which one is better.

Adam Hyperparameter Grid Search Results

β_1	β_2	Validation Loss	Best Epoch	Epochs Run
0.85	**0.990**	**0.0862**	**15**	**25**
0.85	0.995	0.0877	15	25
0.85	0.999	0.0882	22	32
0.90	0.990	0.0885	16	26
0.90	**0.999**	**0.0887**	**18**	**28**
0.90	0.995	0.0889	16	26
0.95	0.990	0.0892	17	27
0.95	0.999	0.0899	24	34
0.95	0.995	0.0905	17	27

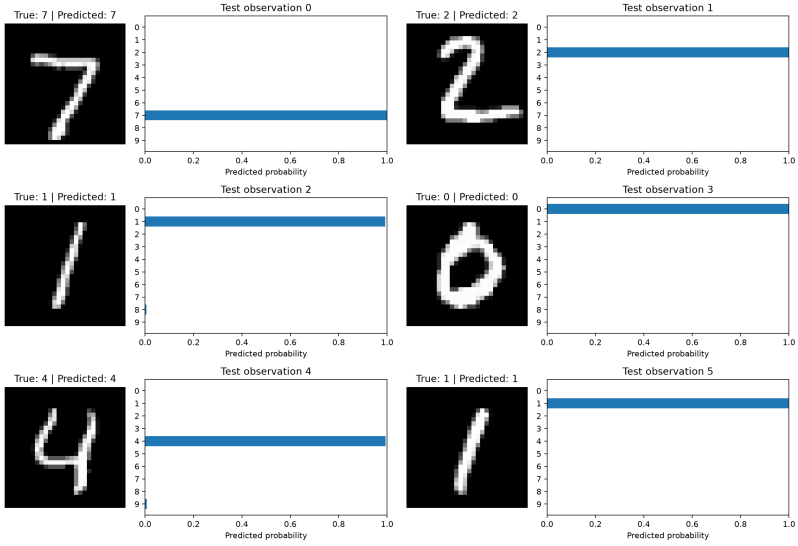
- **Best configuration:** $\beta_1 = 0.85$, $\beta_2 = 0.99$.
- Minimum validation loss: 0.0862, reached at epoch 15.

Comparison of Default ADAM, tuned ADAM, and SGD



Optimizer	SGD	Default ADAM	Tuned ADAM
Accuracy	0.961	0.9745	0.9752

Examples of prediction



GitHub repository

Tutorial code is available on my website:
`https://owenvisser.github.io/`

